

Linked List Part 1: 1D LL and DLL Basics

What is a Linked List ? A linked list is a linear data structure resembling a chain, where each node is connected to the next, and each node represents an individual element. Unlike arrays, the elements in a linked list are not stored in contiguous memory locations. In arrays, adding a new element requires the next memory location to be empty, which cannot always be guaranteed. Therefore, expanding an array beyond its initial size can be challenging and inefficient. This limitation is not present in linked lists, which can dynamically grow and shrink as needed.

A linked list is a data structure containing two crucial pieces of information, the first being the data and the other being the pointer to the next element. The 'head' is the first node, and the 'tail' is the last node in a linked list.

Why Linked List over arrays ? Unlike arrays, the size of the Linked List can be decreased or increased at any location and at any point of time efficiently.

Difference Between Struct and Class in Linked Lists

Aspect Struct Class

Definition A struct is a user-defined data type that groups together different data types to form a single unit. It is primarily used for simple data structures. A class is a blueprint for creating objects, providing more advanced features such as inheritance, encapsulation, and polymorphism. **Access Control** By default, all members of a struct are public. By default, all members of a class are private. **Usage in Linked Lists** Often used for defining the nodes in a simple linked list where each node typically contains data and a pointer to the next node. Used for defining more complex linked list structures that require encapsulation and additional functionality, such as methods for insertion, deletion, and traversal. **Memory Management** Typically used in simpler scenarios with straightforward memory management, often using stack allocation. More suitable for dynamic memory management using heap allocation, allowing for more control over the lifecycle of linked list nodes. **Example**

```

struct Node {
    int data;
    Node* next;
};

class Node {
private:
    int data;
    Node* next;
public:
    Node(int data) : data(data), next(nullptr) {}
    int getData() { return data; }
    Node* getNext() { return next; }
    void setNext(Node* nextNode) { next = nextNode; }
};

```

Node Structures

```cpp

#include <bits/stdc++.h>

using namespace std;

```

struct Node {
 int data;
 Node* next;

 Node(int data1) {
 data = data1;
 next = nullptr;
 }

 Node(int data1, Node* next1) {
 data = data1;
 next = next1;
 }
};

```

# 1. Introduction to Singly LinkedList

---

## Problem

Convert an array into a singly linked list.

## Optimal Approach

Create the head from the first element, then keep attaching new nodes using a mover pointer.

```
Node* convertArrayToSinglyLL(vector<int>& arr) {
 // Hint: head fixed rahega, mover last node par move karega.
 if (arr.empty()) return nullptr;

 Node* head = new Node(arr[0]);
 Node* mover = head;

 int n = arr.size();
 for (int i = 1; i < n; i++) {
 Node* temp = new Node(arr[i]);
 mover->next = temp;
 mover = temp;
 }

 return head;
}
```

## Problem

traversal the linked list

## Optimal Approach

Create the head from the first element, then keep attaching new nodes using a mover pointer.

```

void traversalinSinglyLL(Node* head) {
 // Hint: head fixed rahega, mover last node par move karega.
 if (head==nullptr) return;

 Node* mover = head;
 while(mover){
 cout<<mover->data<<" ";
 mover=mover->next;
 }
 this will print the elements of the linked list

}

```

Time Complexity:  $O(n)$

Space Complexity:  $O(n)$ , because `n` nodes are created

## 4. Find the Length of the Linked List

### Problem

Count the number of nodes in the linked list.

### Optimal Approach

Traverse until `nullptr` and count nodes.

```

int lengthOfSLL(Node* head) {
 // Hint: temp ko aage badhao, count ko saath-saath badhao.
 int count = 0;
 Node* temp = head;

 while (temp != nullptr) {
 count++;
 temp = temp->next;
 }

 return count;
}

```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

## 5. Search in Linked List

---

### Problem

Search whether a key exists in the linked list.

### Optimal Approach

Traverse and compare every node's data with the key.

```
bool searchInSLL(Node* head, int key) {
 // Hint: array ki linear search jaisa, bas index nahi pointer move hota hai.
 Node* temp = head;

 while (temp != nullptr) {
 if (temp->data == key) return true;
 temp = temp->next;
 }

 return false;
}
```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

## 3. Deletion of the Head of LL

---

### Problem

Delete the first node and return the new head.

### Optimal Approach

Move head to `head->next`.

```

Node* deleteHeadSLL(Node* head) {
 // Hint: empty list par direct nullptr return karo.
 if (head == nullptr) return nullptr;

 Node* temp = head;
 head = head->next;
 delete temp; // this will erase the temp;

 return head;
}

```

Time Complexity:  $O(1)$

Space Complexity:  $O(1)$

### 3. Deletion of the Tail of LL

---

#### Problem

Delete the last node and return the new head.

#### Optimal Approach

```

Node * deleteTailSLL(Node* head) {
 if(head==nullptr || head->next==nullptr) return head; // if linkedlist size is 0
 or 1 then return nullptr
 Node*temp= head;
 while(temp->next->next!=nullptr) {
 temp=temp->next;
 }
 delete temp->next;
 temp->next= nullptr;
 return head;
}

```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

### 3. Deletion of the kth position of LL

---

#### Problem

Delete the k node of ll

#### Optimal Approach

```
Node * deleteKthSLL(Node* head ,int k){
 if(head==nullptr)return nullptr;
 if(k==1){// head ko delere kro;
 Node*temp =head;
 head=head->next;
 delete temp;
 return head;
 }

 Node* pre=nullptr;
 Node* temp= head;
 int count= 0;
 while(temp!=nullptr){
 count++;
 if(count==k){
 pre->next=pre->next->next;
 delete temp;
 break;
 }

 pre= temp ;
 temp =temp ->next;
 }
 return head;
}
```

Time Complexity:  $O(K)$

Space Complexity:  $O(1)$

### 3. Deletion of the first ocurence of given element of LL

---

## Problem

Delete the first occurrence of given element of LL;

## Optimal Approach

```
Node * deleteValueSLL(Node* head ,int el){
 if(head==nullptr)return nullptr;
 if(head->data==el){// head ko delere kro;
 Node*temp =head;
 head=head->next;
 delete temp;
 return head;
 }

 Node* pre=nullptr;
 Node* temp= head;

 while(temp!=nullptr){

 if(temp->data==el){
 pre->next=pre->next->next;
 delete temp;
 break;

 }

 pre= temp;
 temp =temp ->next;
 }
 return head;
}
```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$



## 2. Insertion at the tAIL of Linked List

---

### Problem

Insert a new node AT Tail OF LL.

### Optimal Approach

inset the new node at tail of ll;

```
Node* insertAtTailSLL(Node* head, int val){
 if(head==nullptr){
 return new Node(val);
 }
 Node* temp= head;
 while(temp->next!=nullptr){
 temp= temp->next;
 }
 Node* newNode= new Node(val);
 temp->next=newNode;
 return head;
}
```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

## 2. Insertion at before value of Linked List

---

### Problem

Insert before value OF LL.

## Optimal Approach

```
Node* insertbeforeValue(Node* head, int ele, int val){
 if(head==nullptr){
 return nullptr;// linked list is empty;

 }
 if(head->data==val){
 return new Node(ele, head);
 }
 Node* temp= head;

 while(temp!=nullptr){

 if(temp->next->data==val){
 Node* x= new Node(ele, temp->next);
 temp->next=x;
 break;

 }
 temp= temp->next;

 }

 return head;
}
```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

## Introduction to Doubly LL

```
struct Node {
 int data;
 Node* back;
 Node* next;

 Node(int data1) {
 data = data1;
 back = nullptr;
 next = nullptr;
 }

 Node (int data1, Node* next1, Node* back1){
 data = data1;
 back = pre1;
 next = next1;
 }
};
```

## 6. Introduction to Doubly LL

---

### Problem

Convert an array into a doubly linked list.

### Optimal Approach

Maintain both `next` and `back` links.

```

Node* convertArrayToDLL(vector<int>& arr) {
 // Hint: DLL mein new node ka back old tail hota hai.
 if (arr.empty()) return nullptr;

 Node* head = new Node(arr[0]);
 Node* prev = head;

 int n = arr.size();
 for (int i = 1; i < n; i++) {
 Node* temp = new Node(arr[i], nullptr, prev);
 prev->next = temp;
 prev = temp;
 }

 return head;
}

```

Time Complexity:  $O(n)$

Space Complexity:  $O(n)$ , because `n` nodes are created

## 8. Delete Head of Doubly Linked List

### Problem

Delete the first node of a DLL and return the new head.

### Brute / Better

Not required. Head deletion is directly  $O(1)$ .

### Optimal Approach

Move head forward and set new head's `back` to `nullptr`.

```

Node* deleteHeadDLL(Node* head) {
 // Hint: single node case mein nextHead nullptr hoga.
 if (head == nullptr || head->next == nullptr) return nullptr; // if ll is empty of
 1 element

 Node* pre = head;
 head = head->next;
 head->back = nullptr;
 pre->next = nullptr;
 delete pre;
 return head;

}

```

Time Complexity:  $O(1)$

Space Complexity:  $O(1)$

## 8. Delete Tail of Doubly Linked List

---

### Problem

Delete the tail of a DLL

### Brute / Better

Not required. Head deletion is directly  $O(1)$ .

### Optimal Approach

move temp pointer to tail node then `tail->back->next = nullptr` `tail->back = nullptr`;

```

Node* deleteTailDLL(Node* head) {
 // Hint: single node case mein nextHead nullptr hoga.
 if (head == nullptr || head->next== nullptr) return nullptr;// if ll is empty of
1 element

 Node* temp= head;
 while(temp->next!=nullptr){
 temp=temp->next;
 }
 Node* prev=temp->back;
 pre->next=nullptr;
 temp->back= nullptr;
 delete temp;
 return head;
}

```

Time Complexity:  $O(1)$

Space Complexity:  $O(1)$

## 8. Delete kth pos of Doubly Linked List

---

### Problem

Delete the kth of a DLL

## Optimal Approach

```
Node* deleteKthDLL(Node* head, int k) {
 // Hint: single node case mein nextHead nullptr hoga.
 if(head==nullptr) return head;

 Node* temp= head;
 int count=0
 while(temp!=nullptr){
 count++;
 if(count==k){
 break;
 }
 temp=temp->next;
 }
 Node* prev=temp->back; // prev ttemp front
 Node* front=temp->next;
 if(pre==nullptr){
 return deleteHeadDLL(head);
 }
 else if(front==nullptr){
 return deleteTailDLL(head);
 }

 prev->next= front;
 front->back= prev;
 temp->next=nullptr;
 temp->beak=nullptr;
 delete temp;
 return head;

}
```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

## 7. Insert Node Before Head in Doubly Linked List

---

### Problem

Insert a node before the current DLL head.

### Brute / Better

Not required. Head insertion in DLL is  $O(1)$ .

### Optimal Approach

Create a new node, connect it before old head, and update `back`.



```

Node* insertBeforeHeadDLL(Node* head, int value) {
 // Hint: newHead->next = old head, old head->back = newHead.
 if(head==nullptr){
 Node* newHead = new Node(value);
 return newHead;
 }
 Node* newHead = new Node(value,head, nullptr); // data next back
 head->back=newHead;
 return newHead;
}

Node* insertBeforeTailDLL(Node* head, int value) {
 if(head->next==nullptr){
 return insertBeforeHeadDLL(head, value) ;
 }
 Node* temp= head;
 while(temp->next!=nullptr){
 temp= temp->next;
 }
 Node* prev= temp->back;
 Node* newNode=new Node(value,temp, prev);
 prev->next=newNode;
 temp->back=newNode;
 return head;
}

Node* insertBeforekthDLL(Node* head, int value, int k) {
 if(k==1){
 return insertBeforeHeadDLL(head, value);
 }
 int count=0;
 Node* temp= head;
 while(temp!=nullptr){
 count++;
 if(count==k){
 break;
 }
 temp=temp->next;
 }
 Node*prev=temp->back;
 Node* newNode=new Node(value,temp, prev);

```

```

 prev->next=newNode;
 temp->back=newNode;
 return head;
 }
 void insertBeforeNode(Node* temp, int value){// node is newer be the head //ie no in
sertino before the head
 Node*prev=temp->back;
 Node*front=temp->back;
 Node* newNode= new Node(value, temp,prev);
 prev->next= newNode;
 temp->back= newNode;

 }

```

Time Complexity:  $O(n)$

Space Complexity:  $O(1)$

## 9. Reverse a Doubly Linked List

---

### Problem

Reverse a doubly linked list.

### Brute Approach

Store values in a stack, then rewrite node data while traversing again.

```

Node* reverseDLLBrute(Node* head) {
 // Hint: stack LIFO se values reverse order mein milengi.
 stack<int> st;
 Node* temp = head;

 while (temp != nullptr) {
 st.push(temp->data);
 temp = temp->next;
 }

 temp = head;
 while (temp != nullptr) {
 temp->data = st.top();
 st.pop();
 temp = temp->next;
 }

 return head;
}

```

Time Complexity:  $O(2n)$

Space Complexity:  $O(n)$

## Better

No separate better approach needed. Pointer reversal is optimal.

## Optimal Approach

Swap `back` and `next` for every node.

```
Node* reverseDLLOptimal(Node* head) {
 // Hint: har node par back-next swap karo; last processed node new head deta hai.
 if (head == nullptr || head->next == nullptr) return head;

 Node* current = head;
 Node* last = nullptr;

 while (current != nullptr) {
 last = current->back;
 current->back = current->next;
 current->next = last;
 current = current->back;
 }

 return last->back;
}
```

**Time Complexity:**  $O(n)$

**Space Complexity:**  $O(1)$